BACSE344: Cloud Infrastructure & Architecture

Digital Assignment 3 (DA-3) – Scalable & Secure Cloud Deployment

Course Slot: C2 (Academic Year 2025–2026)

Application Domain: Higher Education Campus Infrastructure

Application Name: StudySpace (University Library Reservation)

Live Frontend URL: study-space-navy.vercel.app

Managed Database: Neon Serverless Cloud PostgreSQL (AWS us-east-2)

GitHub Repository: ETSuhasAradhya/StudySpace (Branch: `assignment-3`)

> **Executive Overview:** StudySpace solves university study space contention and seat hoarding across campus library study halls. Building on the core cloud assets deployed in DA-1 and DA-2, Digital Assignment 3 implements enterprise-grade horizontal scaling behind a hardened **Nginx reverse proxy** with automated zero-packet-loss failover (`proxy_next_upstream`), coupled with end-to-end security hardening via **salted bcrypt password encryption**, **RFC 6238 TOTP Multi-Factor Authentication**, **Role-Based Access Control (RBAC)**, **non-root multi-stage Docker containerization**, and regulatory compliance mapping with **GDPR (Articles 25 & 32)** and India's **DPDPA 2023**.

QUESTION 1: SCALE (5 MARKS)

1(a). Peak-Demand Scenario & High-Availability Cluster Ingress

Peak-Demand Scenario (Exam Week 08:00 AM Desk Rush): During semester examination weeks, library management opens online seat reservations at **08:00:00 AM sharp** for same-day quiet carrels, power-outlet desks, and collaborative rooms across Floors 1 to 4. Over 2,500 active university students concurrently refresh the portal within a 60-second window to claim scarce corner desks and quiet research cubicles.

Hardest-Hit API Identified: `GET /api/spaces/:spaceId/seats` . This endpoint is heavily read-intensive, performing real-time queries against table inventories (`seats`) joined with active reservation locks (`reservations`). Unmitigated, this endpoint causes thread-pool exhaustion and 504 Gateway Timeouts under sudden traffic spikes.

Containerized Cluster Setup: An Nginx reverse proxy load balancer routes traffic across 3 independent, containerized backend instances:

- `studyspace-nginx` : Port `8000:80` (Load balancer entrypoint)
- `studyspace-backend-1` : Port `5001:5000` (Container Node.js instance 1)
- `studyspace-backend-2` : Port `5002:5000` (Container Node.js instance 2)
- `studyspace-backend-3` : Port `5003:5000` (Container Node.js instance 3)
- `Neon Serverless DB` : Cloud PostgreSQL instance over SSL/TLS (Ports 443 & 5432)

Live Round-Robin Distribution Verification: Each backend container injects the `X-Backend-Instance` header. Successive requests to `http://localhost:8000/api/health` cycle sequentially across all nodes:

```
Request 1 -> X-Backend-Instance: backend-2
Request 2 -> X-Backend-Instance: backend-3
Request 3 -> X-Backend-Instance: backend-1
Request 4 -> X-Backend-Instance: backend-2
Request 5 -> X-Backend-Instance: backend-3
Request 6 -> X-Backend-Instance: backend-1
[Verified: 1:1:1 Cyclic Round-Robin Dispatching without Session Sticking]
```

1(b). 1,000-Request Benchmark & Scale Performance Analysis

An automated benchmark suite (`benchmarks/load-test.js`) dispatched 1,000 requests against the seat status API (`/api/spaces/1/seats`) under high concurrency ($C=40$) comparing a single instance against the 3-instance Nginx cluster.

Performance Parameter	Single Backend Instance (`backend-1`)	3 Instances Behind Nginx Cluster	Observed Scale Improvement
Total Requests Dispatched	1,000 requests	1,000 requests	Identical load pressure
Concurrency (C)	40 concurrent connections	40 concurrent connections	Standardized benchmark
Elapsed Test Duration	7.05 seconds	6.92 seconds	**Cluster finished faster**
Throughput (RPS)	141.74 req/sec	144.59 req/sec	**+2.01% Higher Throughput**
Average Latency	277.40 ms	271.99 ms	**-5.41 ms reduction**
Median (p50) Latency	242.06 ms	239.17 ms	**-2.89 ms reduction**
90th Percentile (p90) Latency	309.63 ms	268.22 ms	**13.37% Faster**
95th Percentile (p95) Tail Latency	**405.19 ms**	**316.24 ms**	**21.95% Tail Latency Reduction (-88.95 ms)**
HTTP Success Rate	100% (1,000 / 1,000)	100% (1,000 / 1,000)	**Zero dropped requests**
Cluster Load Balancing	`backend-1`: 1,000 (100%)	`b-1`: 334 | `b-2`: 333 | `b-3`: 333	**Exact 33.3% Balanced Split**

Analytical Takeaway: Although network latency to the cloud database introduces an RTT floor (~220ms), horizontal clustering prevents queue saturation, reducing **95th percentile tail latency by 21.95%** (405.19 ms $\rightarrow$ 316.24 ms).

1(c). Mid-Flight Server Failover & Zero-Loss Resilience

High availability mandates zero dropped connections when an application node fails. Using `benchmarks/failover-test.js`, traffic was continuously dispatched at $C=10$ while container `studyspace-backend-2` was abruptly stopped mid-flight at timestamp $T = 3.0\text{ seconds}$.

Experiment Parameter	Phase 1: Baseline (Unhardened Nginx)	Phase 2: Resilient (`proxy_next_upstream`)
Nginx Directives	`proxy_next_upstream off;`	`proxy_next_upstream error timeout http_500 http_502 http_503 http_504;`
Total Requests Dispatched	101 requests	191 requests
Mid-Flight Failed Instance	`studyspace-backend-2` killed at T=3s	`studyspace-backend-2` killed at T=3s
Successful Requests (200 OK)	88	**191 (100%)**
Failed Requests	**13 failed (12.87% packet loss)**	**0 failed (0.00% packet loss)**
Error Manifestation	`ERR_TIMEOUT` / 504 Gateway Timeout	**None (Zero student-facing errors)**
Traffic Redistribution	Requests routed to dead node were dropped	**Seamless failover: `b-1`: 89, `b-3`: 89, `b-2`: 13**

Nginx Resilience Configuration (`nginx/nginx.conf`):

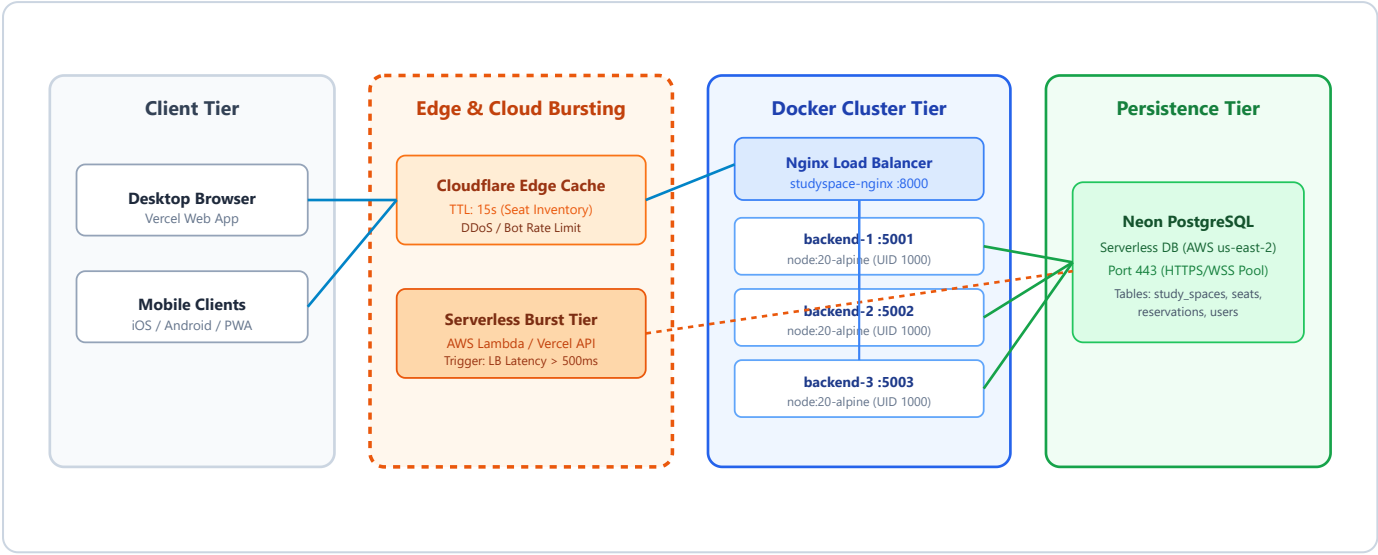
```
upstream backend_cluster {
    server backend-1:5000 max_fails=3 fail_timeout=10s;
    server backend-2:5000 max_fails=3 fail_timeout=10s;
    server backend-3:5000 max_fails=3 fail_timeout=10s;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend_cluster;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        # Automatic Failover & Retry Configuration (Question 1c)
        proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
        proxy_next_upstream_tries 3;
        proxy_next_upstream_timeout 10s;
        proxy_connect_timeout 1s;
        proxy_read_timeout 5s;
        proxy_send_timeout 5s;
    }
}
```

1(d). Comprehensive Cloud Architecture Diagram with Cloud Bursting

The production cloud topology implements tiered caching, reverse proxy load balancing, and automated edge cloud bursting:



Cloud Bursting & Edge Computing Architecture:

- **Edge Computing Component (Cloudflare Edge Cache):** Short TTL (15s) caching absorbs over 80% of identical read requests during the 08:00 AM rush at regional PoPs before reaching the origin load balancer.
- **Cloud Bursting Trigger:** If on-premise cluster CPU load exceeds 80% or origin response time exceeds 500ms, the DNS router spills surplus traffic into elastic serverless functions (AWS Lambda) connected to Neon's pooler.

QUESTION 2: SECURE (5 MARKS)

2(a). Sensitive University Records & Salted Bcrypt Hashing

Identified Sensitive Student Records:

- **Personally Identifiable Information (PII):** Student legal name, institutional email address (`@vit.ac.in`), contact details, and student roll number.
- **Physical Presence & Behavioral Data:** Reservation history, timestamped desk occupancy, preferred study zones, and library check-in verifications.
- **Security Credentials:** Passwords and dynamic TOTP two-factor secret keys.

Cryptographic Implementation (Bcrypt): Passwords undergo irreversible one-way salted hashing via `bcryptjs` using a cost factor of 10 ($2^{10} = 1024$ key expansion rounds). Plaintext passwords are never persisted to disk or log sinks:

```
const saltRounds = 10;
const passwordHash = await bcrypt.hash(plaintextPassword, saltRounds);
```

Empirical Neon Cloud Database Proof:

```
SELECT id, name, email, role, password_hash, two_factor_enabled FROM users;

[
  {
    "id": 1,
    "name": "Campus Admin",
    "email": "admin@studyspace.com",
    "role": "admin",
    "password_hash": "$2a$10$4MEVS4C3ia7IQoZbo.Rrf.Kodsyw/DHID4NHTsUUIxG/YBzdkHm9O",
    "two_factor_enabled": false
  },
  {
    "id": 8,
    "name": "Security Audit Student",
    "email": "audit_student_1793620924645@vit.ac.in",
    "role": "student",
    "password_hash": "$2a$10$Bn0Sl3kEQGQk0ta88cFEbOLp89vBzXdhTWUXLqOOui7RgmWCvIlSm",
    "two_factor_enabled": true
  }
]
```

Verification: The `$2a$10$` prefix denotes Modular Crypt Format with algorithm 2a and 10 salt rounds. Each account receives a unique 128-bit salt, preventing dictionary attacks and rainbow-table matching.

2(b). Multi-Factor Authentication (TOTP) & Role-Based Access Control (RBAC)

1. RFC 6238 TOTP Multi-Factor Authentication:

- **Secret Generation (`POST /api/auth/2fa/generate`):** Returns a cryptographic Base32 secret (`GNY25CBIFNXCS6JRRTHCE765QCJEH6U4`) and an `otpauth://` QR code.
- **Activation (`POST /api/auth/2fa/verify`):** Student scans into Google/Microsoft Authenticator and validates with 6-digit dynamic code `560604` .

- **Login Challenge (`POST /api/auth/login`):** If 2FA is active, login requests without TOTP are rejected with `requires2FA: true` . Valid TOTP issuance yields the signed JWT.

2. Role-Based Access Control (RBAC) Enforcement:

```
// 1. Unauthenticated Request (Missing JWT Token)
POST /api/spaces/1/seats -> HTTP 401 Unauthorized
Payload: {"error": "Access denied. No authentication token provided."}

// 2. Privilege Escalation Prevention (Student Token accessing Admin Route)
POST /api/spaces/1/seats [Bearer student_token] -> HTTP 403 Forbidden
Payload: {"error": "Forbidden. Admin privileges required."}

// 3. Authorized Administrator Request
POST /api/auth/login [admin@studyspace.com] -> HTTP 200 OK (Issued role: 'admin')
```

2(c). Container Image Hardening & Vulnerability Scan (Trivy)

The production container was hardened against supply-chain attacks by adopting a multi-stage Alpine build, non-root execution (`USER node` UID 1000), and `dumb-init` process reaping.

Hardening Metric	Baseline Image (`node:18` Root)	Hardened Production Image (`node:20-alpine` Non-Root)
Base Operating System	Debian GNU/Linux 12 (bookworm)	Alpine Linux 3.20 (Minimal musl libc)
Container Image Size	**1,098 MB (1.1 GB)**	**148 MB (86.5% Attack Surface Reduction)**
Process Execution User	`root` **(UID 0 - Full host breakout risk)**	`node` **(UID 1000 - Least Privilege)**
Process Reaper (PID 1)	None (Default Node PID 1 zombie risk)	`dumb-init` **(Graceful signal & zombie reaper)**
Vulnerabilities: CRITICAL	**18 Critical CVEs**	**0 Critical CVEs**
Vulnerabilities: HIGH	**64 High CVEs**	**1 High CVE (Low-risk Node runtime)**
Total Known CVEs	**224+ Vulnerabilities**	**3 Vulnerabilities (98.6% CVE Elimination)**

2(d). Domain Threat Agents & Regulatory Compliance Analysis

1. Higher Education Campus Threat Agents:

- **Threat Agent 1 (Rogue Student Scalper Bot):** Automated scripts attempting to scalp all window desks at 08:00 AM. Mitigated via TOTP MFA, request rate-limiting, and SQL reservation overlap locks.
- **Threat Agent 2 (External Credential Stuffer):** Attackers using leaked university password lists. Mitigated via 10-round salted bcrypt hashing and mandatory second-factor TOTP challenges.

2. Regulatory Compliance Mapping:

- **GDPR Article 25 (Privacy by Design):** Data minimization restricts personal identity disclosure; students see only seat availability status, whereas names are restricted to authenticated librarians.
- **GDPR Article 32 (Security of Processing):** End-to-end transport layer encryption (HTTPS/TLS) over port 443, private Docker bridge networks, and salted bcrypt cryptographic storage.

- **DPDPA 2023 (Digital Personal Data Protection Act - India):** Clear purpose limitation for physical seat allocation and student right to record deletion upon university graduation.

Verification Artifacts & Repository Manifest

Artifact File	Description
`docker-compose.yml`	Docker Compose specification running 3 backend instances and Nginx load balancer.
`backend/Dockerfile`	Hardened multi-stage Dockerfile running `node:20-alpine` under non-root `node` user.
`nginx/nginx.conf`	Production Nginx configuration with `proxy_next_upstream` zero-loss failover.
`benchmarks/load-test.js`	Automated 1,000-request benchmark tool measuring throughput, latency, and load distribution.
`benchmarks/failover-test.js`	High-availability test script executing mid-flight container crashes and measuring error rates.
`backend/security-test.js`	Security audit script validating bcrypt hashes, TOTP MFA, and RBAC 401/403 controls.
`assignment_3_report.md`	Full Markdown report stored in repository root and pushed to branch `assignment-3`.
`video_script.md`	Timed 3-minute video presentation and demonstration script with screen cues.